

KwaloKWakolê Group

Description du projet — plateforme e-commerce et culture (document généré automatiquement).

1. Vue d'ensemble

Le projet est une application web **Django 6** dédiée à une marketplace afro-moderne : produits locaux, artisans, boutiques, prestataires de services, ainsi qu'un espace **Culture** (musique, artistes, concerts et billets avec QR). L'interface inclut une **PWA** (installation mobile), un assistant conversationnel **Kwa** (FAQ), et des flux de paiement incluant **GeniusPay** pour le e-commerce et les abonnements, avec possibilité de paiement à la livraison.

2. Fonctionnalités principales

- **Boutique** : catalogue, catégories, panier, favoris, commandes clients.
- **Marketplace** : vendeurs et prestataires, tableaux de bord, demandes de service.
- **Abonnements** : formules vendeur/prestataire, montants paramétrables par l'admin, paiement (GeniusPay ou simulation).
- **Culture** : profils artistes, catalogue musical, événements, achat de billets et QR.
- **Comptes** : inscription, django-allauth (Google, Facebook), rôles client / vendeur / prestataire / admin.
- **Paiements commandes** : GeniusPay (checkout) ou espèces à la livraison ; webhooks et pages de retour.
- **Avis, messagerie, notifications** : engagement utilisateurs autour des commandes.

3. Structure technique (applications Django)

Composant	Rôle
kwalo/	Configuration (settings, URLs racine, middleware, static).
store/, catalog/	Vitrine, liste produits, panier, checkout, catégories.
marketplace/	Vendeurs, prestataires, inscriptions, dashboards.
orders/	Commandes et statuts.
payments/	Paiements (GeniusPay, webhooks, retours utilisateur).
subscriptions/	Plans, abonnements, paiements récurrents / Genius.
culture/	Artistes, musique, concerts, billets, QR.
accounts/	Profils, authentification, espace admin métier.
reviews/, messaging/, notifications/	Avis/ fil de discussion, alertes.
content/	Pages éditoriales simples.
templates/, static/	Thème UI, PWA (manifest, service worker), chatbot JS.
frontend/	Assets React / Vite (intégration sélective sur certaines pages).

4. Prérequis & déploiement

Python 3.12+, PostgreSQL en production (dj-database-url), variables d'environnement pour les secrets (SECRET_KEY, DATABASE_URL, clés GeniusPay GENIUS_API_KEY / GENIUS_API_SECRET, webhook GENIUS_WEBHOOK_SECRET, email SMTP). Fichiers statiques via WhiteNoise ; HTTPS et CSRF_TRUSTED_ORIGINS configurés pour le domaine public.

5. Installation locale (rappel)

1. Créer un environnement virtuel et **pip install -r requirements.txt**.
2. Copier **.env.example** vers **.env** et renseigner les variables.
3. Exécuter **python manage.py migrate** puis **python manage.py runserver**.
4. Optionnel : **python manage.py seed_test_data** pour des données de démonstration.

6. Points d'entrée URL (racine)

/admin/ — Administration Django ; **/compte/** — Comptes ; **/vendeur/** — Marketplace ; **/paiement/** — Paiements ; **/abonnement/** — Abonnements ; **/culture/** — Culture ; **/messagerie/**, **/notifications/**, **/avis/** ; **/** — Store (accueil, produits, panier). Fichiers PWA : manifest.webmanifest, sw.js, hors-ligne/.